

CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: Gabe Mankowski

Blane Fullmer

Project Teams Group #: C_07

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

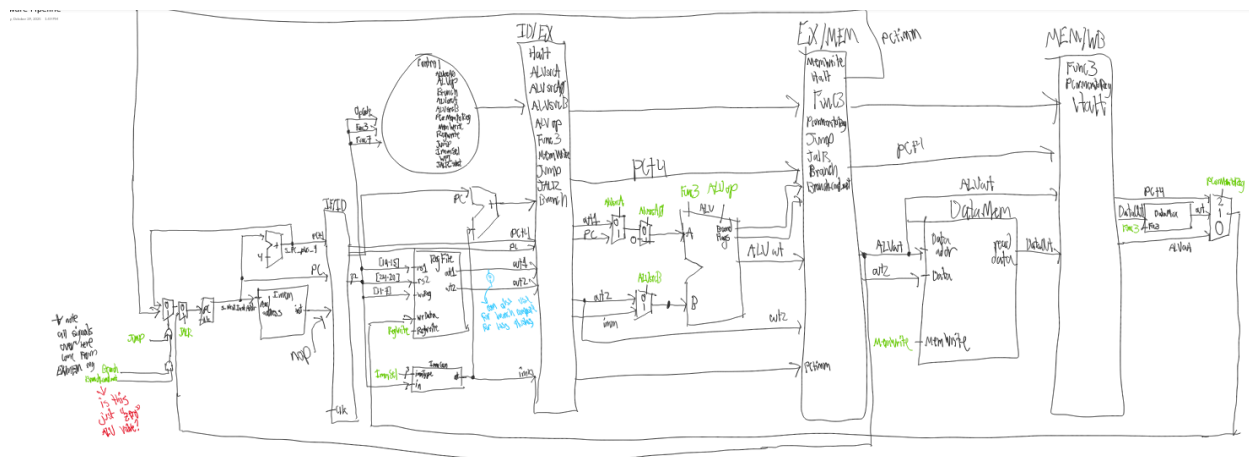
IF/ID: PC+4, PC, Inst

ID/EX: WFI, ALUSrcA, ALUSrcA0, ALUSrcB, ALUOp, MemWrite, RegWrite, Jump, Jalr, Branch, PcorMemtoReg, Func3, PC, PC+4, PC+imm, Out1, Out2, RegWrAddr

EX/MEM: WFI, MemWrite, RegWrite, Func3, PcorMemtoReg, Jump, Jalr, Branch, BranchCondMet, PC+4, ALUout, Out2, PC+imm, RegWrAddr

MEM/WR: WFI, RegWrite, PcorMemtoReg, Func3, RegWrAddr, ALUout, dMemOut, PC+4

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.

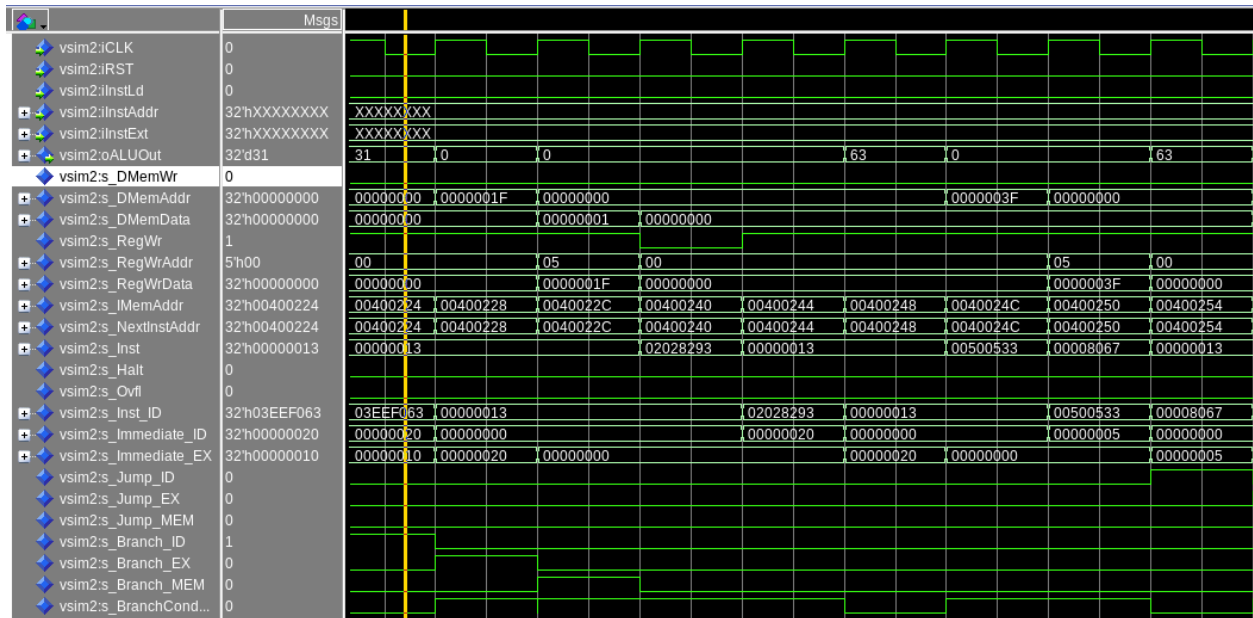
Arithmetic instruction example:

	Msgs								
vsim1:CLK	0								
vsim1:RST	0								
vsim1:InstLd	0								
vsim1:InstAddr	32hXXXXXXXX	XXXXXXXXXX							
vsim1:InstExt	32hXXXXXXXX	XXXXXXXXXX							
vsim1:oALUOut	32hABCDE456	00000000	ABCDE000	00402004	00000000	ABCDE456			
vsim1:s_DMemWr	0								
vsim1:s_DMemAd...	32h00000000	00000000	ABCDE000	00402004	00000000				
vsim1:s_DMemD...	32h00000000	00000000							
vsim1:s_DMemOut	32h11223344	11223344			55667788	11223344			

This shows a LUI 0xABCDE000 (output of ALU), and then an AUIPC 0x2 (adds 2 to the PC which is already +2 from base so that's why it's +4), then we needed a nop to delay since the

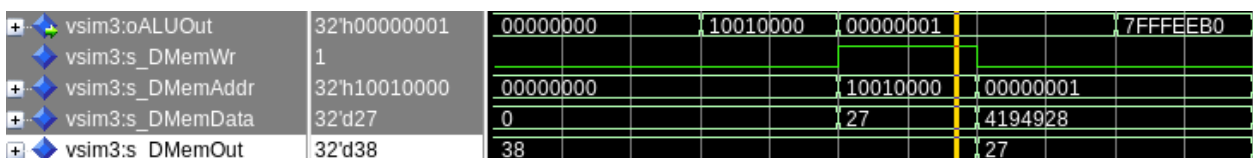
next instruction is a addi that uses the LUI output, which adds 456 (which is shown correctly in output of ALU).

Branch instruction example:



The first “31” is an addi instruction. The Next instruction is a “bgeu” which passes (as shown in “BranchCondMet” signal going to high) since the ALU output is 0 which is $\geq$. The PC then jumps (not shown in screenshot from 0x0040022C to 0x00400240). Then there are 3 NOP since our design requires 3, then it computes an addi (output 63), then we need 2 more NOPS since the next instruction is “mv” which requires the output of that ADDI

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.



*First merge, putting 2 solo arrays together, elements 0 and 1

The first merge is looking at the values of 38 and 27, what is shown in this SS is the 27 being placed first into the 2 sized array since it is a smaller value.

+ vsim3:InstAddr	32'hXXXXXXXX	XXXXXXXX			
+ vsim3:InstExt	32'hXXXXXXXX	XXXXXXXX			
+ vsim3:oALUOut	32'h00000001	000000...	10010004	00000001	
vsim3:s_DMemWr	1				
+ vsim3:s_DMemAddr	32'h10010004	00000000		10010004	
+ vsim3:s_DMemData	32'd38	0		38	
+ vsim3:s_DMemOut	32'd27	27			

*second part

It is now placing the 38 after the 27 in DMEM, this completes the first recursive merge.

+ vsim3:InstAddr	32'hXXXXXXXX	XXXXXXXX			
+ vsim3:InstExt	32'hXXXXXXXX	XXXXXXXX			
+ vsim3:oALUOut	32'h00000001		10010008	00000001	
vsim3:s_DMemWr	1				
+ vsim3:s_DMemAddr	32'h10010008	00000000		10010008	
+ vsim3:s_DMemData	32'd3	0		3	
+ vsim3:s_DMemOut	32'd43	27		43	

*Second merge, looking at 43 and 3, elements 2 and 3

This is looking at the 43 and 3 elements in the array, it is placing 3, since it is smaller, in first

+ vsim3:InstAddr	32'hXXXXXXXX	XXXXXXXX			
+ vsim3:InstExt	32'hXXXXXXXX	XXXXXXXX			
+ vsim3:oALUOut	32'h00000001	00000000	1001000C	00000001	
vsim3:s_DMemWr	1				
+ vsim3:s_DMemAddr	32'h1001000C	00000000		1001000C	
+ vsim3:s_DMemData	32'd43	0		43	
+ vsim3:s_DMemOut	32'd3	27		3	

*second part

It is now putting the 43 in next to the 3, which completes the second merge. So currently it has merged 2 parts together and has [27,38] and [3,43] in memory.

[1.d] Report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

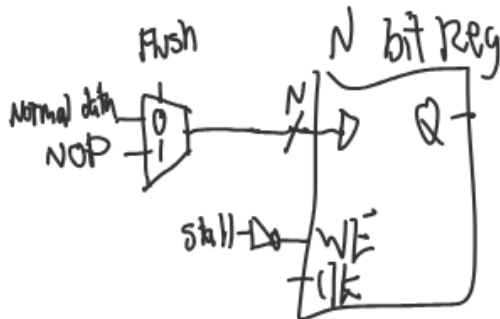
FMax: 53.25mhz

Critical path is Memory Load

Path: MEM/WB pipeline register -> WB_mux -> RegFile

[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.

Stall/Flush Example



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.

Wave - Default		Msgs	
s_IFID_i_PCPlus4	32h1000000C	XXXXXXXX	10000004
o_Q	32h10000008	00000000	XXXXXXXX 10000004 00000000 10000008
s_IDEX_i_PCPlus4	32h10000008	00000000	XXXXXXXX 10000004 00000000 10000008
s_IDEX_o_PCPlus4	32h10000008	00000000	XXXXXXXX 10000004 00000000 10000008
s_EXMEM_i_PCPlus4	32h10000008	00000000	XXXXXXXX 10000004 00000000 10000008
s_MEMWB_i_PCPlus4	32h10000008	00000000	XXXXXXXX 10000004 00000000 10000008

This is an example that shows the signal being propagated through all of the pipeline stages. The input to the IFID PC+4 gets set to 0x4 -> next cycle sent to IDEX -> IFID is reset to 0 -> 0x0 is propagated through, then a few cycles happen and the input to the IFID is set to 0x0C but the register is stalled so it doesn't take the new input and is never propagated down

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

Add - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Addi - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 And - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Andi - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Lui - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Lw - ALUout_EX, ALUout_MEM->DMemAddr, DMemOut, DMemOut_WB, RegWrData
 Xor - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Xori - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Or - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Ori - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Slt - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Slti - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Sltiu - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData

Sll - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Srl - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Sra - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Sw - ALUout_EX, ALUout_MEM->DMemAddr and s_RegData2_ID, RegData2_EX, RegRegData2_MEM->s_DMemData
 Sub - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Jal - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData,
 Jalr - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData,
 Lb - ALUout_EX, ALUout_MEM->DMemAddr, DMemOut, DMemOut_WB, RegWrData
 Lh - ALUout_EX, ALUout_MEM->DMemAddr, DMemOut, DMemOut_WB, RegWrData
 Lbu - ALUout_EX, ALUout_MEM->DMemAddr, DMemOut, DMemOut_WB, RegWrData
 Lhu - ALUout_EX, ALUout_MEM->DMemAddr, DMemOut, DMemOut_WB, RegWrData
 Sli - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Srli - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Srai - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData
 Auipc - ALUout_EX, ALUout_MEM, ALUout_WB, RegWrData

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

Add - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Addi - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 And - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Andi - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Lui - Immediate_ID, Immediate_EX
 Lw - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Xor - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Xori - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Or - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Ori - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Slt - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Slti - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Sltiu - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Sll - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Srl - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Sra - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Sw - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX, Immediate_ID, Immediate_EX
 Sub - RegData1_ID, RegData1_EX, RegData2_ID, RegData2_EX
 Jal - Immediate_ID, Immediate_EX, CurrPC_ID, CurrPC_EX
 Jalr - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX, CurrPC_ID, CurrPC_EX
 Lb - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX Lh - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Lbu - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Lhu - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Sli - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Srli - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Srai - RegData1_ID, RegData1_EX, Immediate_ID, Immediate_EX
 Auipc - Immediate_ID, Immediate_EX, CurrPC_ID, CurrPC_EX

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

Ex/Mem hazard: instruction N produces ALUout_MEM, instruction N+1 consumes RegData1_EX (which is the output of ALUout_MEM)

Ex/Mem hazard: instruction N produces ALUout_MEM, instruction N+1 consumes RegData2_EX (which is the output of ALUout_MEM)

Mem/Wb hazard: instruction N produces ALUout_WB, instruction N+1 consumes RegData1_EX (which is the output of ALUout_WB)

Mem/Wb hazard: instruction N produces ALUout_WB, instruction N+1 consumes RegData2_EX (which is the output of ALUout_WB)

These first 4 can be forwarded.

Load/Use hazard: instruction N is a LW (that needs to get through the MEM/WB register), and the N+1 instruction consumes that value -> must insert bubble in between so its N+1 NOP N.

Branch/Use hazard: Since Branching is computed in the ID stage, you can't forward values to that stage, so if previous instructions are required for that branch to compute, you need to stall the branch.

These last 2 must be stalled.

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF/ID: PC+4, PC, Inst, Flush, WE

ID/EX: WFI, ALUSrcA, ALUSrcA0, ALUSrcB, ALUOp, MemWrite, RegWrite, Jump, Jalr, Branch, PcorMemtoReg, Func3, PC, PC+4, PC+imm, Out1, Out2, RegWrAddr, RegWrite, Flush

EX/MEM: WFI, MemWrite, RegWrite, Func3, PcorMemtoReg, Jump, Jalr, Branch, BranchCondMet, PC+4, ALUout, Out2, PC+imm, RegWrAddr, RegWrite

MEM/WR: WFI, RegWrite, PcorMemtoReg, Func3, RegWrAddr, ALUout, dMemOut, PC+4, RegWrAddr, RegWrite

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

Jal, Jalr, Beq, Bne, Blt, Bge, Bltu, and Bgeu are all resolved in the ID stage

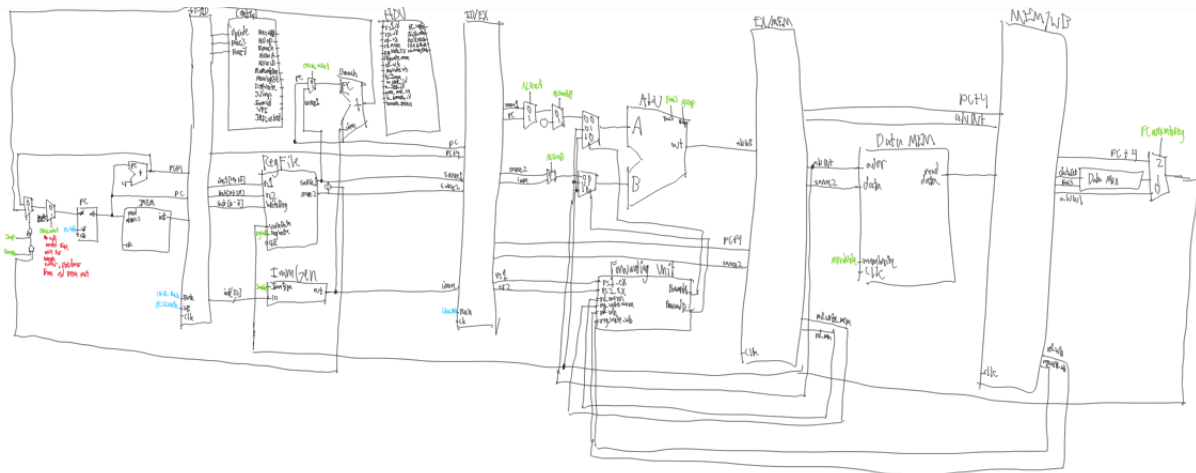
[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

Jal: Flush IF/ID register

Jalr: Stall IF/ID register, if rs1 is written to in earlier instructions, Flush IF/ID

Branches: Stall IF/ID register, Flush IF/ID

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

this doesn't have any task in the lab doc, just says to add QuesaSim Screenshots

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Test #	Test Description	Output	Justification
1	Ex to Ex forwarding	30	15+15 verifies that the forwarding unit correctly forwards data from EX/MEM to ID/EX
2	Mem to Ex forwarding	8	4+4 verifies that the data is being forwarded from MEM/WB to ID/EX
3	Load use stall	16	8+8 verifies that the HDU detects the dependency on the load instruction and inserts a bubble
4	Branch stall (EX)	1	1=pass verifies that the HDU stalls the branch in ID while branch op being calculated in EX

5	Branch stall (MEM)	1	1=pass verifies that HDU stalls branch while op in MEM
6	Double hazard priority	6	3+3 verifies that the forward unit forwards the most recent value (EX) over (MEM / older) version
7	Store data forward	154	77+77 verifies that the value in EX/MEM is forwarded to Alue input B and that forwarded value is then sent to EX/MEM pipeline reg as the rs2 value

Wave - Default		
	Msgs	
clk	1	
addr	10'h000	000
data	32'd0	30
we	0	
q	32'h0000001E	00000000

Test 1

Wave - Default		
	Msgs	
clk	1	
addr	10'h000	001
data	32'd0	8
we	0	
q	32'h0000001E	00000000

Test 2

	Msgs	
clk	1	
addr	10'h000	002
data	32'd0	16
we	0	
q	32'h0000001E	00000000

Test 3

	Msgs	
clk	1	
addr	10'h000	003
data	32'd0	1
we	0	
q	32'h0000001E	00000000

Test 4

	Msgs	
clk	1	
addr	10'h000	004
data	32'd0	1
we	0	
q	32'h0000001E	00000000

Test 5

005	000	013	026	006
6	0		77	154
00000000	0000001E	00000000		

Tests 6/7 (i don't want to hear any jokes)

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Test #	Test Desc	Output	Justification
1	JAL Flush	1	Verifies that PC+4 is correctly flushed
2	JALR Flush	1	Verifies that PC+4 is correctly flushed
3	BEQ Taken Flush	1	Verifies that Branch correctly flushes
4	BNE Taken Flush	1	Verifies that Branch correctly flushes
5	Branch not taken	1	Verifies that pipeline doesn't flush on branch not taken
6	JALR w/ Data hazard	1	Verifies that load/use hazard and IF/ID flush work together
7	Stress	1	Verifies pipeline can flush then do another hazard right after

	Msgs	
vsim1:clk	1	
vsim1:addr	10'h002	000
vsim1:data	32'h00000000	00000001
vsim1:we	0	
vsim1:q	32'h00000000	00000000

Test 1

